## *Guia de Sintaxe Java*

**Classes abstratas, interfaces e polimorfismo**

- extends: declara herança.
- abstract class: classe que não pode ser instanciada diretamente.
- abstract: método sem implementação, que deve ser implementado por uma subclasse concreta.
- @Override: indica sobrescrita de método.
- implements: declara que uma classe implementa uma interface.

**Exceções**

Disparo de exceções:

```
if (valor < 0) {
        throw new IllegalArgumentException(
        "Valor inválido");
}
```

Captura de exceções:

```
try {
        int x = 10 / n;
} catch (ArithmeticException e) {
        System.out.println("Erro aritmético");
} finally {
        System.out.println("Fim");
}
```

- try: bloco no qual uma exceção pode ocorrer.
- catch: trata uma exceção compatível com o tipo declarado.
- finally: executa normalmente após o try e os catch, havendo ou não exceção.
- throw: lança uma exceção.
- throws: declara exceções que um método pode propagar.

**Generics**

```
class Caixa<T> {
        private T valor;
        public Caixa(T valor) {
        this.valor = valor;
        }
        public T get() { return valor; }
        public void set(T valor) {
        this.valor = valor;
        }
}
```

```
Caixa<String> c = new Caixa<>("Java");
String s = c.get();
```

Método genérico

```
static <T> T primeiro(T a, T b) {
        return a;
}
```

**Java Collections Framework**

**Collection — interface geral de coleções**

Usar Collection<E> como parâmetro permite aceitar diferentes implementações, sem exigir operações específicas de List.

**List — interface de lista ordenada por posição**

```
List<String> nomes = new ArrayList<>();
```

- List<E>: mantém elementos em posições e permite repetições.
- ArrayList<E> e LinkedList<E> são implementações de List.
- add(e): insere no final.
- add(i, e): insere na posição i.
- set(i, e): substitui o elemento na posição i.
- get(i): consulta a posição.
- size(): quantidade de elementos.
- contains(e): verifica se contém o elemento.

**Map — interface de mapa que faz associação entre chaves e valores**

```
Map<String, Integer> notas = new HashMap<>();
```

- Map<K,V>: associa chaves (K) a valores (V).
- As chaves são únicas; put() atualiza o valor de uma chave existente.
- get(k): consulta o valor associado.
- containsKey(k): verifica se a chave existe.
- remove(k): remove a associação.
- size(): quantidade de associações.

**Arquivos**

**Streams para arquivos sequenciais de bytes**

```
BufferedWriter out = new BufferedWriter(new
FileWriter("dados.txt"));
out.write("Ana"); out.newLine(); out.close();
BufferedReader in = new BufferedReader(new
FileReader("dados.txt"));
String linha;
while ((linha = in.readLine()) != null)
   System.out.println(linha);
in.close();
```

**Streams para arquivos sequenciais de objetos**

```
// Gravar
ObjectOutputStream out = new ObjectOutputStream(
   new FileOutputStream("dados.dat"));
out.writeObject(objeto);
out.close();

// Ler
ObjectInputStream in = new ObjectInputStream(
   new FileInputStream("dados.dat"));
Tipo objeto = (Tipo) in.readObject();
in.close();
```

**Serializable — serialização padrão**

```
class Aluno implements Serializable {
   private static final long serialVersionUID = 1L;
   String nome;
}
// Gravar: ObjectOutputStream + writeObject(objeto)
// Ler: ObjectInputStream + (Aluno) readObject()
```

**Externalizable — serialização personalizada**

```
class Aluno implements Externalizable {
   String nome;
   public Aluno() {} // construtor público sem argumentos
   public void writeExternal(ObjectOutput out) throws
IOException {
      out.writeObject(nome);
   }
   public void readExternal(ObjectInput in)
      throws IOException, ClassNotFoundException {
      nome = (String) in.readObject();
   }
}
```

Serializable: grava automaticamente o estado serializável.
Externalizable: exige implementar writeExternal() e readExternal(); a ordem de leitura deve corresponder à de escrita.